

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФГБОУ ВО Волгоградский государственный технический университет
Кафедра «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ
АВТОМАТИЗИРОВАННЫХ СИСТЕМ»

*Перебазирование и редактирование истории кода в распределенных
системах контроля версий*

Методические указания



Волгоград
2018

Рецензент

к. т. н., доц. Н.П. Садовникова

Издается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Перебазирование и редактирование истории кода в распределенных системах контроля версий: метод. указания / сост. О. А. Сычев, Г.В. Терехов; ВолгГТУ. – Волгоград, 2018. – 18 с.

В методических указаниях изложены цели, содержание и порядок выполнения лабораторных работ «Перебазирование веток» и «Редактирование истории кода» по дисциплинам «Эволюция и сопровождение программного обеспечения» и «Распределенный контроль версий кода», приведены основные сведения о назначении и использовании команд перебазирования и редактирования истории кода в современных системах распределенного контроля версий Git и Mercurial.

Методические указания предназначены для изучения дисциплин «Эволюция и сопровождение программного обеспечения» и «Распределенный контроль версий кода». Методические указания могут быть использованы студентами, обучающимися по направлению 09.03.04 «Программная инженерия», а также широким кругом лиц, интересующихся программированием.

© Волгоградский государственный
технический университет, 2018

Оглавление

1. Перезагрузка изменений.....4
2. Назначение и особенности использования редактирования истории..10

1. Перебазирование изменений

Назовем базой коммита предыдущий, то есть родительский коммит. Базой для ветки будет, соответственно, коммит, после которого эта ветка отделилась от главной. Перебазирование – это изменение базы, то есть перенос ветки (начиная с какого-то коммита и до конца) на другое место, чаще всего вперед по линии коммитов. Обратите внимание, что перебазирование изменяет историю коммитов и, если ваш репозиторий до перебазирования был кем-то вытянут (и он делал на нем свою работу), то этот разработчик получит проблемы с обновлением после перебазирования. Поэтому перебазирование лучше употреблять только если вы уверены, что вашу работу никто не затягивал.

Наиболее частое употребление перебазирования заключается в перебазировании локальных изменений до выталкивания их на сервер. Предположим, вы вытянули изменения коллег, провели свою работу и хотите ее вытолкнуть, но ее необходимо слить с коммитами людей, работавших параллельно с вами. При использовании команды `merge` мы получим дополнительный коммит и разветвляющуюся историю – там, где, возможно, никакого особого смысла в ветках нет – они образовались просто случайно из-за вашей одновременной работы. При наличии в команде более двух разработчиков ситуация становится еще хуже, особенно при отсутствии строгой дисциплины в отношении того, кто и в каком порядке сливает изменения. Три разработчика, сделав по одному рабочему (с новым кодом (см. рис. 1)) коммиту, бессистемно их сливая способны создать показанную на рис. 2 сетку из 9 коммитов (цветом выделены коммиты-слияния). Такую историю очень трудно читать и отлаживать, используя `bisect`.

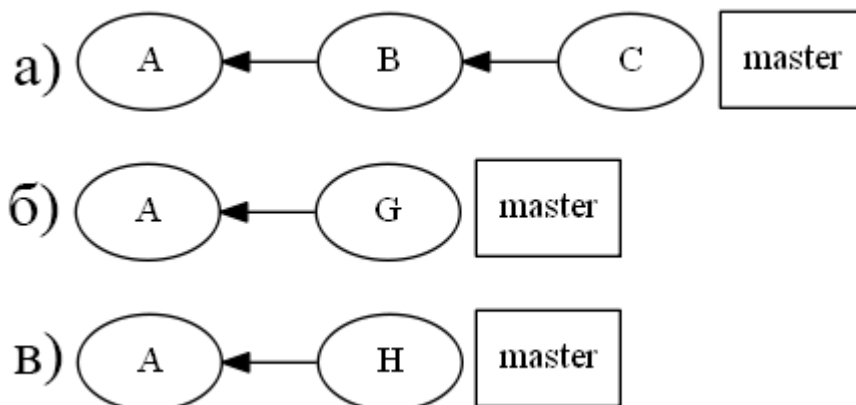


Рисунок 1. Пример трех репозиториях до слияния

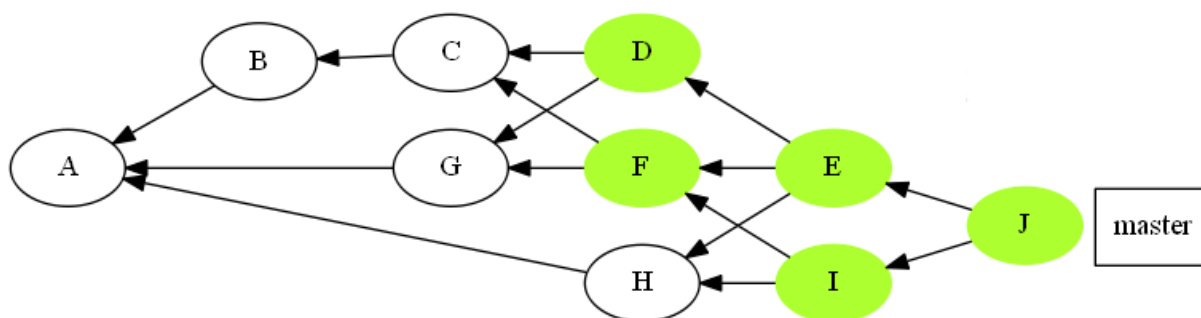


Рисунок 2. Пример репозитория с бессистемным слиянием

Однако в тех же условиях историю можно сохранить линейной, с тремя коммитами. Для этого при затягивании изменений можно использовать команду `pull --rebase`, которая затянет изменения с указанного репозитория и перебазирует ваши коммиты на их вершину, как будто вы работали последовательно. Это абсолютно безопасно для репозитория если перед началом своей работы вы сделали `pull` с того же сервера, так как перебазируются локальные коммиты, которые никуда не выталкивались. Если три программиста в примере выше использовали бы `pull --rebase`, история кода выглядела бы как на рисунке 3.



Рисунок 3. Пример репозитория с использованием перебазирования

Второй частый случай применения `rebase` заключается в перераспределении работы в системе начальник-подчиненный. Как уже говорилось выше, в главе про ветки, выполнение слияния с продакшн-ветками (т.е. теми, в которых формируются будущие релизы) – это реальная власть в коллективе программистов, и делегировать ее часто бывает нежелательно. Однако слияние может включать нудную и тщательную работу по разрешению конфликтов, на которую у начальника не всегда хватает времени. В некоторых случаях, конечно, разрешением конфликтов должен заниматься именно самый опытный разработчик (начальник), но бывает достаточно большое количество простых, но трудоемких в разрешении конфликтов (особенно при слиянии выправлений стиля кодирования и т.д.), которые могут быть разрешены простыми разработчиками. Перебазирование позволяет начальнику, не отдавая права на слияние, переложить разрешение конфликтов на подчиненных. В этом случае подчиненный выполняет перебазирование своей ветки перед окончательным слиянием, а начальнику останется лишь проверить (используя `diff` между ветками и автоматические тесты), что слияние ничего не испортило и запустить команду слияния без конфликтов. В этом случае подчиненному может прийти применять

`push --force`, так как он уже выталкивал свои изменения раньше для обзора начальнику. Безопасность таких изменений истории гарантируется корпоративной культурой: никто кроме начальника не должен вытягивать изменения у его подчиненного, а сам начальник не должен делать поверх них свои коммиты, если он хочет, чтобы изменения были перебазированы подчиненным.

Еще один типичный пример использования перебазирования – когда вы работаете длительное время над веткой с новыми возможностями (feature) и вам нужно периодически добавлять туда новые доработки из master (или default). Использование `merge` в таких условиях может создать большое количество дополнительных коммитов-слияний в вашей ветке, в то время как перебазирование ее на вершину master будет обновлять код в ней без дополнительных коммитов. Однако такое перебазирование следует делать осторожно, тестируя результат после каждого перебазирования (особенно с конфликтами): отсутствие дополнительных коммитов-слияний не позволит вам обнаружить ошибку, внесенную неудачным слиянием, с помощью `bisect`.

Самый частый и простой случай использования команды `rebase` в Git и Mercurial это указание ее опцией команды `pull`: `pull --rebase`. В этом случае команда вытягивает изменения в текущей (или указанной) ветке и перебазирует все локальные (не вытолкнутые на сервер) изменения в ней поверх вытянутых. Обычно такая команда выполняется перед выталкиванием и служит для поддержки линейной истории кода.

Команда `git rebase` понимает задание на перебазирование через три основных параметра в указанном порядке: приемник (куда перебазировать, задается через опцию `--onto`, по умолчанию принимается как вершина апстрима), апстрим (ветка, являющаяся базовой для перебазирования – набор перебазируемых коммитов начинает отсчитываться с ответвления перебазируемой ветки от апстрима) и собственно перебазируемую ветку (по умолчанию – текущая ветка).

Классическое перебазирование в виде переноса точки ответвления перебазируемой ветки на вершину ветки апстрима, получается если переключиться на перебазируемую ветку командой `git checkout` и указать в параметрах `git rebase` только ветку апстрима (см. рис. 4-6).

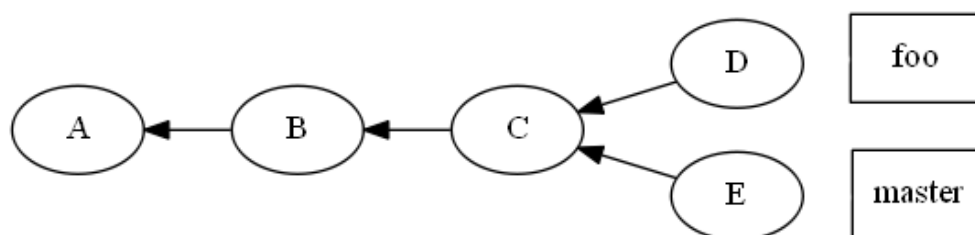


Рисунок 4. Пример состояния репозитория до перебазирования

```
$ git checkout foo
```

```
$ git rebase master
First, rewinding head to replay your work on top of it...
Applying: added staged command
```

Рисунок 5. Пример классического перебазирования в git

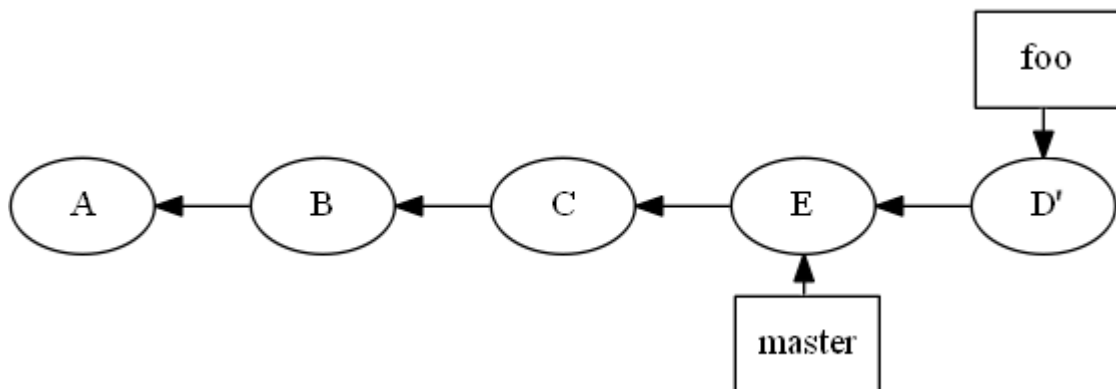


Рисунок 6. Пример состояния репозитория после перебазирования

Если перебазируемая ветка ранее выталкивалась на сервер, то для ее выталкивания после перебазирования понадобится использовать опцию `--force`. При этом надо следить, чтобы выталкивать таким образом только одну ветку – если вы забудете указать имя выталкиваемой ветки и вытолкните их все, вы можете стереть незатянутые к себе коммиты в других ветках.

Использование опции `--onto` с явным указанием приемника позволяет переводить при перебазировании более сложные перемещения, например, изменить точку ответвления на другую ветку.

В качестве примера рассмотрим репозиторий с тремя ветками: `master`, `feature1` и `feature2` (см. рис. 7). В ветке `feature1` находятся сложные функции в процессе реализации, которые еще рано добавлять в `master`, а в ветке `feature2` реализовано небольшое улучшение, которое уже готово к интеграции в `master`. Для этого необходимо перебазировать `feature2` на вершину `master`, как показано на рисунке 8. Результирующий репозиторий показан на рисунке 9.

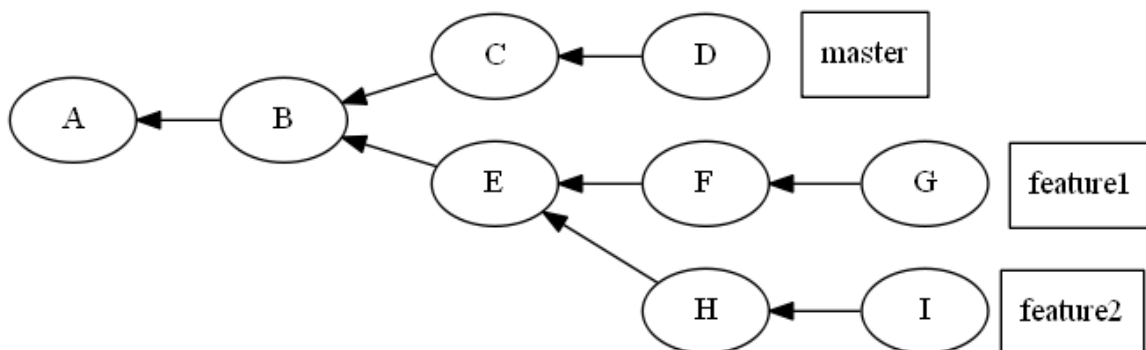


Рисунок 7. Пример Git репозитория с тремя ветками до перебазирования
`git rebase --onto master feature1 feature2`

Рисунок 8. Перебазирование ветки feature2 на вершину master с использованием опции --onto

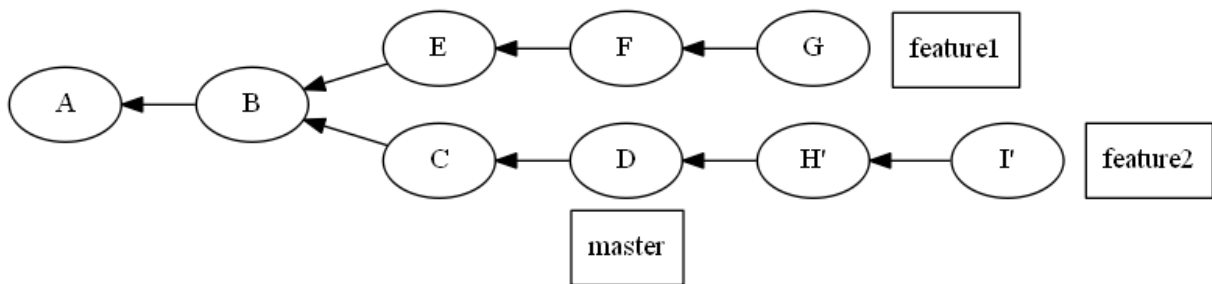


Рисунок 9. Пример git репозитория после перебазирования

Также с помощью `git rebase --onto` можно удалить коммиты в середине ветки (см. рис. 10-12), если все три параметра ссылаются на коммиты в одной ветке и приемник расположен раньше апстрима. Однако такое удаление требует большой аккуратности при вводе команды и не рекомендуется в общем случае, подобных результаты можно достичь легче используя `git rebase --interactive`.



Рисунок 10. Пример git репозитория до перебазирования

```
git rebase --onto master~2 master~1 master
```

Рисунок 11. Перебазирование в git, приводящее к удалению коммита C



Рисунок 12. Результат перебазирования для удаления коммитов

В Mercurial команда `rebase` является расширением (поставляемым в стандартной инсталляции) и должна быть специально включена в файле настроек перед началом работы, как показано на рисунке 13.

```
[extensions]
rebase =
```

Рисунок 13. Пример строки из файла настроек для включения перебазирования

Команда `hg rebase` использует другую систему опций для определения, что и куда переносить. Коммит-приемник (куда переносить) указывается с помощью опции `-d` (`--dest`), по умолчанию это верхний коммит в текущей именованной ветке. Существует две возможности указать ветку для перебазирования: база или коммит-источник. Указание базы (опции `-b`, `--base`) менее точно, но более удобно: в этом случае перебазирование использует всю ветку от базы до первого коммита,

ответвившего ее от общего потомка с приемником. Это наиболее типичный случай перебазирувания и использование базы позволяет избавить пользователя от ручного поиска коммита, начавшего перебазируемую ветку. По умолчанию базой принимается коммит, являющийся предком текущего содержания рабочего каталога.

Допустим, у нас имеется репозиторий, состояние которого показано на рисунке 14. Необходимо, как и в примере, описанном выше, выполнить перебазирувание ветки `feature2` в ветку `default`. Для начала нужно переключиться на последний коммит `I` из множества коммитов ветки `feature2`, затем перебазировать всю ветку после коммита `D` ветки `default`, как показано на рисунке 15 а). Тоже самое можно сделать с помощью одной команды, достаточно указать после ключа `--base` последний коммит ветки `feature2` (см. рис. 15 б). Также можно указать источник – коммит, с которого начинать перебазирование при помощи опции `--source` (см. рис. 15 в). Результирующее состояние репозитория показано на рисунке 16.

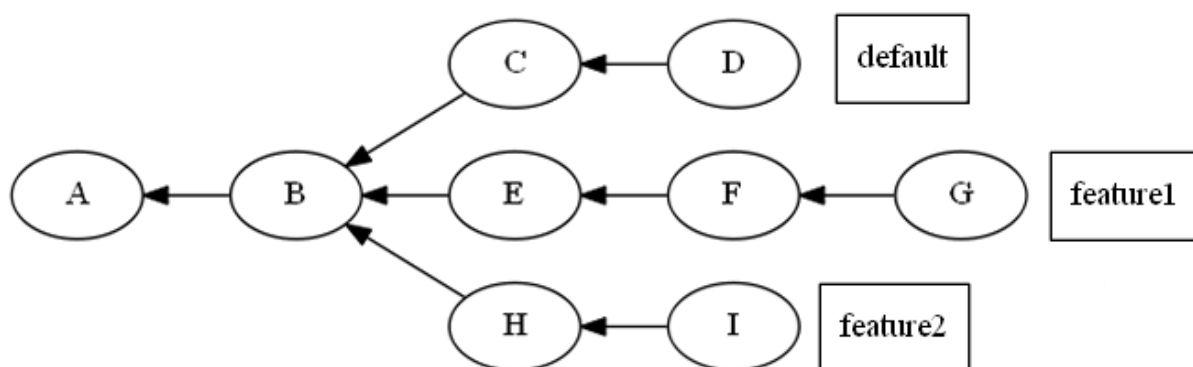


Рисунок 14. Пример Mercurial репозитория с тремя ветками до перебазирувания

<code>\$ hg update I</code>	<code>\$ hg rebase</code>	<code>\$ hg rebase</code>
<code>\$ hg rebase --dest D</code>	<code>--dest D --base I</code>	<code>--dest D --source</code>
		<code>H</code>
a)	б)	в)

Рисунок 15. Перебазирование в Mercurial

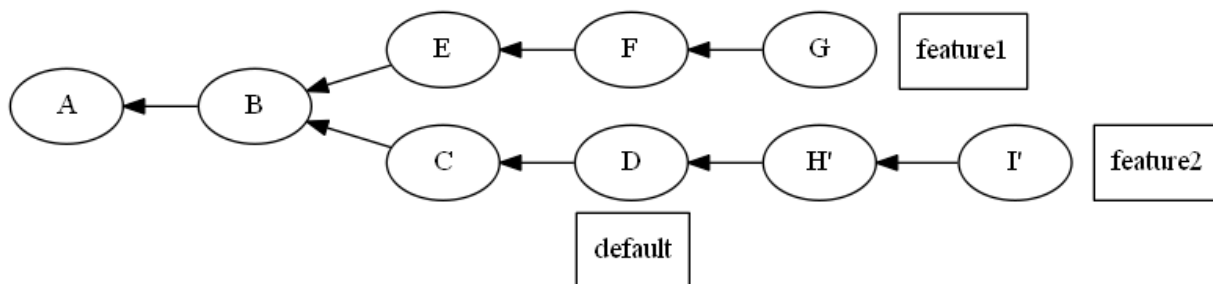


Рисунок 16. Пример Mercurial репозитория после перебазирувания

Если вам нужно перебазировать не всю ветку, а ее часть, то можно использовать только опцию `-s` (`--source`): в этом случае указывается точный коммит, с которого и начнется перебазирование. Допустим, у нас

есть репозиторий, состояние которого показано на рисунке 17. Необходимо перебазировать коммиты F и G после коммита D, а коммиты E оставить ответвленным от B, как есть сейчас (см. рис. 18). Результирующий репозиторий показан на рисунке 19.

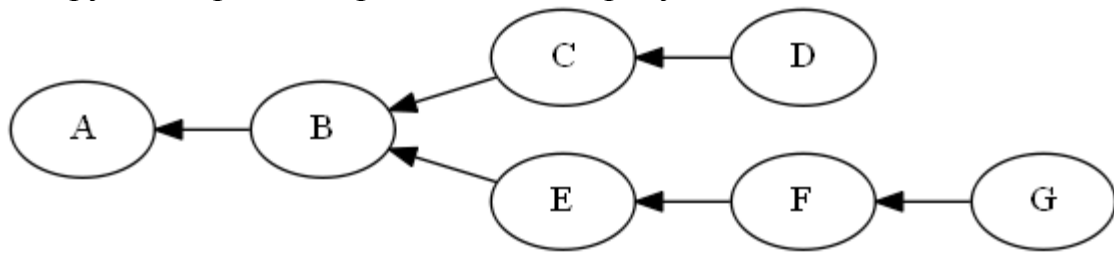


Рисунок 17. Пример Mercurial репозитория с двумя ветками до перебазирования

```
$ hg rebase -s F -d D
```

Рисунок 18. Перебазирование в Mercurial с указанием источника

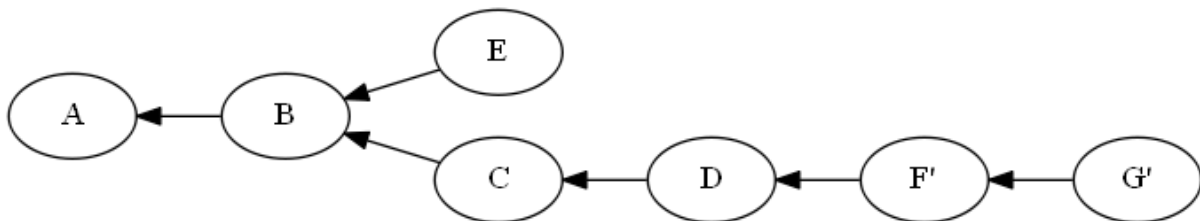


Рисунок 19. Пример Mercurial репозитория после перебазирования

2. Назначение и особенности использования редактирования истории

В повседневной работе часто делаются неудачные коммиты: от случаев коммита ненужных файлов или отладочной печати до слишком мелких и незапускающихся (потому что программист хотел регулярно бекапить свою работу) или наоборот очень крупных и неатомарных коммитов. Особенно часто это происходит при наличии в команде неопытных программистов или новичков. Да и самому бывает стыдно вытаскивать сделанный в спешке неудачный коммит и его исправление дополнительным коммитом.

Существует две стратегии отношения к истории проекта. Простейшая стратегия «как есть» подразумевает сохранение истории проекта именно как истории – со всеми ее проблемами. Такой подход по-своему логичен, но неудобен для анализа истории кода с помощью `bisect`, `blame` и подобных инструментов, а также, например, при переносе изменений с помощью `cherry-pick`. Вторая стратегия подразумевает подчистку

неудач и хранение идеализированной истории. Она требует, чтобы после завершения реализации какого-либо подпроекта (исправление ошибки, реализация новых функций) коммиты были исправлены так, чтобы представить идеализированную ситуацию разработки. Это делается либо разработчиком перед выталкиванием на сервер, либо (при наличии иерархии) после просмотра кода начальником (без слияния его до исправления недостатков).

Многие формы такого редактирования можно симулировать с помощью различных команд системы контроля версий, правильно определив значения параметров (см. удаление коммитов с помощью перебазирования, например), однако такой подход неудобен и связан с большим риском сделать ошибку. Поэтому системы контроля версий предусматривают специальные команды для редактирования истории изменений.

В Git редактирование истории называется интерактивным перебазируанием и вызывается командой `rebase -i` или `rebase --interactive`. При этом, чтобы определить диапазон коммитов для редактирования, надо указать либо ветку, относительно которой делается перебазирование (тогда редактироваться будут все коммиты в текущей ветке после ответвления от указанной), либо указать диапазон коммитов. Так если вы находитесь в ветке `feature`, то команда `git rebase -i master` позволит вам отредактировать все коммиты после ответвления `feature` от `master`. Команда `git rebase -i HEAD~5` даст вам отредактировать последние 5 коммитов.

При выполнении этой команды Git вызывает текстовый редактор со специальным временным файлом, представляющим собой план редактирования: список коммитов, перед каждым из которых находится команда `pick`. Коммиты находятся в том порядке, в котором они производились (т.е. с точностью до наоборот по отношению к результатам команды `log`). В файле также выводится закомментированная справка. Ваша задача на этом этапе – отредактировать файл так, чтобы он содержал план желаемых исправлений.

Вы можете удалять строки (что приведет к удалению соответствующих коммитов со всем содержимым) и переставлять их (меняя местами коммиты). Перестановка коммитов может оказаться очень полезной, если вы заметили ошибку в каком-то коммите не сразу (или ее заметил ваш начальник, а коммит был не последним), так что исправляющий ошибку коммит находится не сразу за ошибочным, что усложняет их объединение. Также на коммиты можно влиять, изменяя команды перед ними (см. рис. 20-21). Возможные команды приведены в таблице 1.

Таблица 1. Команды интерактивного перебазирувания Git

Команда	Описание
pick	Стандартная команда, применяет коммит в указанном месте.
reword	Сохранить коммит, но дать отредактировать коммит-сообщение.
edit	Остановится перед применением этого коммита, давая возможность отредактировать его содержание (т.е. код программы) вручную. Может быть использована для разбиения большого коммита на несколько.
squash	Объединить этот коммит с предыдущим. Команда открывает текстовый редактор с коммит-сообщениями всех объединяемых коммитов чтобы дать вам сделать новое коммит-сообщение.
fixup	Объединить этот коммит с предыдущим, отбросив его коммит-сообщение. Обычно применяется в случае, если объединяемый коммит исправляет ошибки в предыдущем.
exec	Позволяет запустить команду операционной.

Допустим, у нас есть репозиторий, чье состояние изображено на рисунке 20.

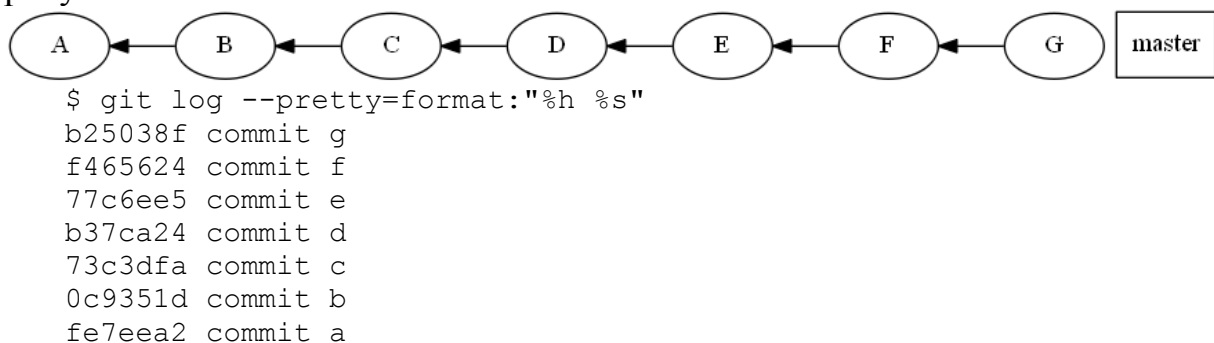


Рисунок 20. Пример репозитория для редактирования истории

Запустим редактирование коммитов, как показано на рисунке 21.

```
$ git rebase -i HEAD~6
```

Рисунок 21. Запуск редактирования последних шести коммитов

После запуска `rebase -i` откроется текстовый редактор с командами, которые необходимо применить к коммитам (см. рис. 22). Допустим, нам нужно оставить как есть коммит А, заменить коммит сообщение коммита В, отредактировать коммит С, объединить коммит D с F, и объединить коммит E с G, отбросив при этом коммит сообщение коммита G. Отредактируем его, как показано на рисунке 23.

```

pick 0c9351d commit b
pick 73c3dfa commit c
pick b37ca24 commit d
pick 77c6ee5 commit e
pick f465624 commit f
pick b25038f commit g

```

Рисунок 22. Пример файла после использования `git rebase -i`

Изменим верхнюю часть файла на список команд, показанный на рисунке 23.

```

reword 0c9351d commit b
edit 73c3dfa commit c
pick b37ca24 commit d
squash f465624 commit f
pick 77c6ee5 commit e
fixup b25038f commit g

```

Рисунок 23. Пример отредактированного файла с инструкциями для `git rebase -i`

После того, как вы введете новый план, сохраните файл и выйдете из редактора, система контроля версий отменяет все указанные коммиты (сохраняя их содержимое во временной зоне) и начинает применять их заново в соответствии с вашим планом. Команды `reword` и `squash` откроют текстовый редактор для редактирования коммит-сообщения.

В нашем примере сначала сработает инструкция `reword`, вводим новое коммит сообщение и сохраняем изменения (см. рис. 24).

```

[detached HEAD db90798] edited message commit b
1 file changed, 1 insertion(+), 1 deletion(-)

```

Рисунок 24. Пример работы инструкции `reword`

Если вы ввели команду `edit` то система остановится после этого коммита и выдаст инструкции по изменению коммита. Вы можете использовать `git commit --amend` чтобы отредактировать сообщение и сам коммит, после чего продолжить перебазирование командой `git rebase --continue` (которая воспроизведет исправленный коммит).

Следующим в нашем примере будет как раз инструкция `edit` (см. рис. 25).

```

Stopped at 73c3dfa... commit c
You can amend the commit now, with
  git commit --amend
Once you are satisfied with your changes, run
  git rebase --continue

```

Рисунок 25. Пример работы инструкции `edit`

После того, как отредактируем текущие изменения, вводим `git commit --amend`, редактируем коммит сообщение, вводим `git rebase --continue` чтобы продолжить редактирование истории. Следующей командой будет `squash`, откроется текстовый редактор, предлагающий отредактировать коммит-сообщение, как показано на рисунке 26.

```

# This is a combination of 2 commits.
# The first commit's message is:
commit d
# This is the 2nd commit message:
commit f
# Please enter the commit message for your changes. Lines
starting

```

```
# with '#' will be ignored, and an empty message aborts the
commit.
# Not currently on any branch.
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   index.php
```

Рисунок 26. Пример работы инструкции squash

После редактирования коммита увидим следующее (см. рис. 27).

```
$ git rebase --continue
[detached HEAD 7fda761] commit d
 1 file changed, 1 insertion(+), 1 deletion(-)
[detached HEAD 9a4c59a] commit f
 1 file changed, 1 insertion(+), 1 deletion(-)
Successfully rebased and updated refs/heads/master.
```

Рисунок 27. Пример окончания перебазирувания

В результате редактирования истории кода будет получен репозиторий, показанный на рисунке 28.



```
$ git log --pretty=format:"%h %s"
9a4c59a commit e
7fda761 commit df
2f94a70 commit c
db90798 edited message commit b
fe7eea2 commit a
```

Рисунок 28. Результат редактирования истории

Если вы использовали `edit` с целью разбить один коммит на несколько, возможностей `git commit --amend` уже недостаточно. Вы должны начать с отмены этого коммита без очистки содержимого файлов (т.е. в `mixed` режиме) выполнив `git reset HEAD^`. Далее вы можете добавлять изменения в индекс и производить коммиты в любом количестве. Когда вы сделаете все необходимые коммиты, используйте `git rebase --continue` для продолжения перебазирувания.

Для демонстрации разбиения одного коммита на несколько, запустим перебазирувание для предыдущего примера (см. рис 20), как показано на рисунке 29.

```
$ git rebase -i HEAD~3
```

Рисунок 29. Пример разбиения одного коммита на три с использованием инструкции `edit`

Затем изменяем инструкции таким образом, чтобы отредактировать предпоследний коммит (см. рис. 30).

```
pick 0c9351d commit b
edit 73c3dfa commit c
```

```
pick b37ca24 commit d
```

Рисунок 30. Пример инструкций для редактирования коммита в режиме интерактивного перебазирувания

В тоге получим вывод, показанный на рисунке 31.

```
Stopped at 0c9351d... commit b
You can amend the commit now, with
  git commit --amend
Once you are satisfied with your changes, run
  git rebase --continue
```

Рисунок 31. Пример вывода результата после редактирования коммита

Сначала отменяем текущий коммит с сохранением изменений при помощи команды `git reset HEAD^`. Наш коммит содержит три ханка, разобьём коммит на три (под каждый ханк свой коммит) при помощи команды `git add -p` (см. рис. 32). Повторим команды `git add -p`, выбрав опцию `y` для добавления в индекс текущего ханка и `git commit -m` трижды, для каждого нового коммита. Как только все изменения будут зафиксированны, продолжим перебазирование командой `git rebase -continue`.

```
$ git reset HEAD^
Unstaged changes after reset:
M index.php

$ git add -p
diff --git a/index.php b/index.php
index 9cd06d5..d32e5ad 100644
--- a/index.php
+++ b/index.php
@@ -1,6 +1,5 @@
...
Stage this hunk [y,n,q,a,d,/,j,J,g,e,]? y
@@ -17,7 +16,6 @@
...
Stage this hunk [y,n,q,a,d,/,K,g,e,]? n
@@ -39,6 +37,6 @@
...
Stage this hunk [y,n,q,a,d,/,K,g,e,]? n

$ git status
# Not currently on any branch.
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# modified:   index.php
#
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in
working directory)
#
# modified:   index.php
```

```

#

$ git commit -m 'commit c1'
[detached HEAD bf6c8c0] commit c1
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git status
# Not currently on any branch.
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in
working directory)
#
# modified:   index.php
#
no changes added to commit (use "git add" and/or "git commit -a")

$ git add -p
diff --git a/index.php b/index.php
index 9cd06d5..d32e5ad 100644
--- a/index.php
+++ b/index.php
@@ -17,7 +16,6 @@
...
Stage this hunk [y,n,q,a,d,/,K,g,e,]? y
@@ -39,6 +37,6 @@
...
Stage this hunk [y,n,q,a,d,/,K,g,e,]? n

$ git commit -m 'commit c2'
[detached HEAD bf6c8c0] commit c2
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git commit -a -m 'commit c3'
[detached HEAD bf6c8c0] commit c3
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git status
# Not currently on any branch.
nothing to commit (working directory clean)

$ git rebase --continue
Successfully rebased and updated refs/heads/master.

$ git log --pretty=format:"%h %s"
b37ca24 commit d
2d1b1dd commit c3
05154a8 commit c2
0d81691 commit c1
0c9351d commit b
fe7eea2 commit a

```

Рисунок 32. Разбиения предпоследнего коммита

Стоит отметить, что у команды `git add -p` есть несколько вариантов дальнейших действий. Если вы введете `?`, Git отобразит, что именно вы можете сделать (см. рис. 33).

```
Добавить в индекс эту часть [y,n,a,d,/,j,J,g,e,?]? ?
y - добавить в индекс эту часть
n - не добавлять в индекс эту часть
a - добавить в индекс эту и все оставшиеся в этом файле части
d - не добавлять в индекс эту и все оставшиеся в этом файле части
g - перейти к некоторой части файла (g - показывает список частей
и затем выполняет переход, g<N> - перейти к части N)
/ - найти часть, соответствующую регулярному выражению
j - отложить принятие решения по этой части, перейти к следующей
части, решение по которой не принято
J - отложить принятие решения по этой части, перейти к следующей
части
k - отложить принятие решения по этой части, перейти к предыдущей
части, решение по которой не принято
K - отложить принятие решения по этой части, перейти к предыдущей
части
s - разбить текущую часть на части меньшего размера
e - вручную отредактировать текущую часть
? - отобразить помощь
```

Рисунок 33. Варианты действий команды `git add -p`

В системе контроля версий Mercurial редактирование истории коммитов осуществляется с помощью расширения `histedit`. Как и любое расширение, оно должно быть явно включено в файле настроек для использования (см. рис. 34).

```
[extensions]
histedit =
```

Рисунок 34. Пример включения расширения `histedit`

Вы можете передать команде `histedit` хеш коммита, начиная с которого вы хотите редактировать историю (команда откажется работать, если встретит опубликованный коммит). Чаще всего эту команду удобнее запускать в виде `hg histedit --outgoing`, что позволяет вам отредактировать все коммиты, которые еще не были отправлены на сервер (т.е. те, которые вытолкнет очередной `push`). После чего вы также получаете для редактирования файл с планом редактирования истории в виде перечня коммитов с командами. Вы можете перемещать строчки коммитов для изменения их порядка. Набор команд немного отличается от интерактивного перебаазирования Git (см. таблицу 2)

Таблица 2. Команды редактирования истории Mercurial

Команда	Описание
<code>pick</code>	Использовать коммит.
<code>mess</code>	Отредактировать сообщение коммита.
<code>edit</code>	Отредактировать содержание коммита. Может использоваться для

	разбиения коммита на несколько.
fold	Слить коммит с предыдущим, аналогично squash в git.
drop	Удалить коммит.

Редактирование коммита в Mercurial осуществляется немного по-другому: вам не нужно его отменять. Вы получаете файлы с содержимым коммита и можете их редактировать, а также делать дополнительные коммиты. Для частичного коммита ханков файла можно воспользоваться визуальным интерфейсом или расширением record. После завершения редактирования, при выполнении `hg histedit --continue` оставшиеся изменения создадут коммит, чье сообщение по умолчанию будет совпадать с сообщением редактируемого коммита.

Обратите внимание, что в Mercurial для удаления коммита надо явно использовать команду `drop`, а не просто удалить строчку. Это страхует от случайных малозаметных удалений, что важно – восстановить удаленный коммит будет невозможно.

При выполнении редактирования истории коммитов следует быть очень внимательным, так как после ее завершения предыдущее состояние стирается, и вернуться к нему будет невозможно. Если редактирование пошло не так, вы можете прервать его с помощью `git rebase --abort` или `hg histedit --abort` и начать заново.